Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 01: Introduction to Threads & Mutual Exclusion

Module:	CS4533 – Concurrent Programming
Lab:	Lab 01 – Introduction to Threads & Mutual Exclusion
Duration:	2 hours (in-class)
Topics:	Thread creation, race conditions, mutual exclusion, Amdahl's law

---

Learning Objectives

By the end of this lab, you should be able to:

- Create and manage POSIX threads (pthreads) in C
- Observe and explain race conditions in shared-memory programs
- Implement mutual exclusion using pthread mutexes
- Reason about the impact of synchronization on performance using Amdahl's law

---

Prerequisites

- A Linux environment with GCC installed
- Basic knowledge of C programming (pointers, functions, structs)
- Compile with: gcc -pthread -o output source.c

Exercise 1: Thread Creation and Joining

Background

A thread is an independent flow of execution within a process. Unlike processes, threads share the same address space, making communication easy but also introducing the possibility of data races. The POSIX Threads (pthreads) library provides the standard API for thread management in C on Unix-like systems.

Task

Write a C program that creates N worker threads (where N is provided as a command-line argument). Each thread should print its thread ID and a unique index number, then return the square of its index as its exit value. The main thread must join all threads and print the returned values.

Requirements

- Use pthread_create() to spawn threads and pthread_join() to collect results.
- Pass each thread its index via the arg parameter (handle pointer casting correctly).

- The program must work correctly for N = 1, 4, 8, and 16.
- Print output must clearly show which thread produced which result.

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *worker(void *arg) {
    // TODO: Extract thread index from arg
    // TODO: Print thread ID and index
    // TODO: Return square of index
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <num_threads>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    // TODO: Create n threads, join them, print results
    return 0;
}
```

Questions

Q1.1: Run your program multiple times with N=8. Do the threads always print in the same order? Explain why or why not.

Q1.2: What happens if you omit the pthread_join() calls? Try it and describe the behaviour.

Exercise 2: Observing Race Conditions

Background

A race condition occurs when multiple threads access shared data concurrently and at least one access is a write. The outcome depends on the unpredictable interleaving of thread execution, leading to incorrect or non-deterministic results.

Task

Write a program where multiple threads increment a shared global counter. Each of the N threads should increment the counter M times (e.g., M = 100,000). The expected final value is N × M, but due to race conditions, the observed value will often be less.

Requirements

- Declare a global variable: int counter = 0;
- Each thread increments counter in a loop M times (no synchronization).
- After joining all threads, print the expected and actual values of counter.
- Run the program at least 10 times and record the results.

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define M 100000

int counter = 0;  // shared variable

void *increment(void *arg) {
    for (int i = 0; i < M; i++) {
        counter++;  // NOT atomic!
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    int n = atoi(argv[1]);
    // TODO: Create n threads running increment
    // TODO: Join all threads
    printf("Expected: %d, Actual: %d\n", n * M, counter);
    return 0;
}
```

Questions

Q2.1: Present your results from 10 runs in a table. What is the range of values you observed? Calculate the error percentage for each run.

Q2.2: Explain at the instruction level (load, modify, store) how counter++ can produce incorrect results with two concurrent threads. Draw a timeline showing an interleaving that causes a lost update.

Q2.3: Does increasing N make the problem worse? Test with N = 2, 4, 8, 16 and report your findings.

Exercise 3: Fixing with Mutex

Background

A mutex (mutual exclusion lock) ensures that only one thread can execute a critical section at a time. In pthreads, you use pthread_mutex_lock() and pthread_mutex_unlock() to protect shared data. This eliminates race conditions but introduces synchronization overhead.

Task

Modify your solution from Exercise 2 to use a pthread mutex to protect the shared counter. Then implement a second version that uses a local counter per thread, combining the results at the end with a single mutex-protected addition.

Part A — Global Mutex

- Add a pthread_mutex_t to protect every increment of counter.
- Verify the final count is always exactly N × M.
- Measure execution time using clock_gettime(CLOCK_MONOTONIC, ...).

Part B — Local Accumulation

- Each thread maintains a local counter and increments it M times (no lock needed).
- After the loop, each thread acquires the mutex once to add its local count to the global counter.
- Compare the execution time of Part A vs Part B.

Timing Code Snippet

```
#include <time.h>

struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
// ... work ...
clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed = (end.tv_sec - start.tv_sec)
              + (end.tv_nsec - start.tv_nsec) / 1e9;
printf("Time: %.6f seconds\n", elapsed);
```

Questions

Q3.1: Present a table comparing execution times for Part A and Part B with N = 1, 2, 4, 8, 16 threads and M = 100,000. Explain the performance difference.

Q3.2: Why is Part B significantly faster? Relate your answer to the concept of lock contention.

Q3.3: What would happen if you forgot to unlock the mutex in the critical section? Test it and describe the result.

Exercise 4: Amdahl's Law Analysis

Background

Amdahl's law states that the maximum speedup of a program using N processors is limited by the fraction of the program that must execute sequentially. If P is the parallelisable fraction, the maximum speedup is: S(N) = 1 / ((1 - P) + P/N). This means even with infinite processors, the serial portion fundamentally limits performance.

Task

Write a program that simulates a workload with both a serial and a parallel component. The serial part sorts an array, and the parallel part processes the sorted data in chunks across multiple threads.

Requirements

- Create an array of 10,000,000 random integers.
- Serial phase: sort the array using qsort().
- Parallel phase: divide the sorted array into N equal chunks. Each thread computes the sum of its chunk.
- Time the serial and parallel phases separately.
- Run with N = 1, 2, 4, 8 and calculate speedup relative to the single-threaded version.

Questions

Q4.1: Present a table showing serial time, parallel time, total time, and speedup for each value of N.

Q4.2: From your measurements, estimate P (the parallelisable fraction). Show your working.

Q4.3: According to Amdahl's law with your estimated P, what would be the maximum theoretical speedup with 64 processors? With infinite processors? Compare with your observed speedups.

Q4.4: Suggest one way to increase P (the parallelisable fraction) for this program. Implement it if possible and measure the impact.

Submission Guidelines

- Submit all source files (.c) along with a report (.pdf) containing your answers, tables, and analysis as z zip file.
- The report should include screenshots of compilation and execution where relevant.
- Name your files as: Lab01_<IndexNumber1_IndexNumber2>.zip.
- Submit via Moodle before the deadline.
- Code must compile without errors using: gcc -pthread -O0 -o output source.c

---